

Perceptrons and activation functions

Module 5 - Lesson 1

Overview

Neural networks begin with the perceptron: a weighted sum of inputs passed through an activation function. Stacking these units with non-linear activations builds deep networks.

Key Concepts

- A neuron computes $z = \sum(w_i * x_i) + b$ and passes it through an activation.
- Activation functions introduce non-linearity; without them, layers collapse to one.
- Sigmoid/tanh squash outputs; ReLU keeps positive inputs and is the common default.
- Softmax turns raw scores into class probabilities for classification.
- Layers connect: input layer, hidden layers, output layer.
- Weights are learned via backpropagation and gradient descent.

Example

```
import numpy as np

def relu(z):
    return np.maximum(0, z)

def layer(x, w, b):
    return relu(x @ w + b)

x = np.array([[0.1, 0.5, 0.3]])
w = np.random.randn(3, 4) * 0.1
b = np.zeros(4)
hidden = layer(x, w, b)
print(hidden)
```

Summary

Neurons compute weighted sums passed through activations. Non-linear activations like ReLU let networks express complex functions, and layered composition is what makes them deep.

Practice Checklist

- Implement a single neuron and compute its output for a few inputs.
- Compare sigmoid and ReLU on the same input values.
- Explain why removing all activations would collapse the network.